JavaScript Lab — Practical 12 (VA 12)

Course: JavaScript Programming Laboratory (Semester V)
Practical No.: 12 (VA 12)
Aim: Build an interactive real-time web app using JavaScript and jQuery with DOM, events, storage, and AJAX integration.
Student Name: Samruddhi Kalbande
PRN: 24070521278
Institution: Symbiosis Institute of Technology (SIT), Nagpur

📌 Aim & Objectives

1. Develop a full-featured, interactive real-time web application uniting **DOM manipulation**, **event handling**, **client-side storage**, and **AJAX integration**.
2. Implement **jQuery Event Delegation** to manage dynamically added DOM elements with high runtime efficiency.
3. Synchronize application state persistently using the **Web Storage API** (`localStorage`).
4. Execute external asynchronous network requests with `$.ajax()` to fetch and render RESTful JSON feeds.
5. Create modern case studies: an informational innovation portal (Smart India Hackathon) and a full-featured e-commerce shopping cart system utilizing ES6 `Map` collections and accessible modal panels.

📂 Project Structure

```
pract 12/
├── README.md                    # Comprehensive Practical 12 documentation
├── README.pdf                   # PDF export of laboratory documentation
├── PRACTICAL/                   # Live Task Manager & AJAX Feed App
│   ├── index.html               # App layout with tasks, statistics, clock & posts
container
│   ├── script.js                # jQuery logic, event delegation, localStorage & AJAX
calls
│   └── style.css                # Task styling, badge counters, buttons & card layouts
├── CASE STUDY 1/                # Smart India Hackathon (SIH) Innovation Portal
│   └── index.html               # Multi-section responsive portal with sticky navigation
& cards
```

```
└── CASE STUDY 2/                  # Rakhi & Rose E-Commerce Store
    ├── index.html                 # Interactive catalog with ES6 Map cart engine & sliding
drawer
    └── images/
        └── rakhi-silver-cc0.jpg   # Product imagery asset
```

🔬 Theoretical Background

1. jQuery Event Delegation

When interactive items are created dynamically at runtime, traditional direct event listeners cannot bind to non-existent nodes. Event delegation solves this by attaching a single listener to a static parent container:

```
// Listens to clicks on .complete buttons inside #taskList, even if added dynamically
$("#taskList").on("click", ".complete", function () {
    const id = Number($(this).closest("li").attr("data-id"));
    // Update task state...
});
```

2. Full-Stack Client Orchestration

A unified modern web application integrates five distinct pillars: - **DOM Engine:** Responsive node rendering and live attribute binding. - **Event Dispatcher:** Unified handling for form submissions, clicks, and transitions. - **Persistent Storage:** Browser `localStorage` retains state between user sessions. - **Interval Timers:** Real-time clock and state polling via `setInterval()`. - **AJAX Pipeline:** Asynchronous HTTP client communicating with third-party web services.

💻 Section 1: Core Practical — Live Task Manager & AJAX Post Reader

Key Features & Code Architecture

- **Persistent Data Store:** State is loaded from `localStorage` on boot (`JSON.parse(localStorage.getItem("tasks")) || []`) and saved on every mutation.
- **Dynamic Task Management:** Users can add new tasks with unique timestamps (`Date.now()`), toggle completion states (adding `.done` classes and modifying button text between "Complete" and "Undo"), or remove items.
- **Real-Time Task Analytics:** Automatically calculates and displays real-time statistics:
- Total task count (`tasks.length`)
- Completed task count (`tasks.filter(t => t.completed).length`)

- **Live Digital Clock:** `updateClock()` runs continuously on a 1-second interval (`setInterval(updateClock, 1000)`).
- **Asynchronous AJAX Ingestion:** Fetches remote articles from an external REST API endpoint: ```javascript $("#loadPosts").on("click", function () { $("#status").text("Loading posts..."); $("#posts").empty();

  $.ajax({ url: "https://dummyjson.com/posts?limit=5", method: "GET", dataType: "json", success: function (data) { $("#status").text("English sample posts loaded successfully!"); data.posts.forEach(function (post) { const div = $("

  ").addClass("post"); $("

  ").text(post.title).appendTo(div); $("

  ").text(post.body).appendTo(div); $("#posts").append(div); }); }, error: function () { $("#status").text("Error loading posts from remote API."); } }); }); ```

🏢 Section 2: Case Studies

Case Study 1: Smart India Hackathon (SIH) Innovation Portal

- **Semantic Structure:** Implements accessible HTML5 landmarks (`<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<footer>`).
- **Responsive Layout:** Adaptive navigation bar and grid systems that reflow from three-column layouts on desktop to clean vertical stacks on mobile.
- **Interactive Information Tracks:** Highlights problem-solving phases ("Understand", "Collaborate", "Create"), achievement callouts, and institutional links.

Case Study 2: Rakhi & Rose E-Commerce Store

- **ES6 Map Data Structure for Cart:** Uses JavaScript `Map()` for $O(1)$ product lookups, insertions, and quantity increments: `javascript const cart = new Map(); // Adding or incrementing item const item = cart.get(id); if (item) { item.quantity += 1; } else { cart.set(id, { id, name, price, quantity: 1 }); }`
- **Real-Time Financial Calculations:** Computes total cart item counts and subtotal/grand totals dynamically using `reduce()` :
- $\text{Total Items} = \sum \text{item.quantity}$
- $\text{Grand Total} = \sum (\text{item.price} \times \text{item.quantity})$
- **Accessible Slide-In Drawer:** Cart panel opens smoothly with `scrollIntoView()` , supports keyboard focus transitions, and toggles `aria-expanded` attributes.

🎯 Learning Outcomes

- Gained end-to-end expertise uniting DOM, events, asynchronous AJAX, and Web Storage in a single reactive application.
- Mastered jQuery event delegation for dynamic, memory-efficient UI components.
- Applied ES6 collections (`Map`) and functional higher-order methods (`reduce` , `filter`) for e-commerce state management.
- Built accessible, responsive user interfaces following web accessibility best practices.